

Unit 3. Classification and Evaluation Metrics

Business Data Analytics: Introductory Machine Learning

From real-valued to binary outcomes

Most business decisions are binary: churn, default, click, purchase. The workhorse model is logistic regression, but the harder question is how to evaluate it.

- **Continuous targets:** R^2 and MSE usually suffice.
- **Binary targets:** require diverse metrics (accuracy, precision, recall, ROC, PR), each answering a different business question.
- **The hidden trap:** class imbalance silently distorts most of these metrics.

1 Logistic Regression

1.1 Probability and information-theoretic preliminaries

Logistic regression is built from three pieces of probabilistic vocabulary: the Bernoulli likelihood, maximum likelihood estimation, and the Kullback–Leibler divergence.

Definition 3.1 (Bernoulli random variable). A random variable Y taking values in $\{0, 1\}$ is

Bernoulli with parameter $\mu \in [0, 1]$, written $Y \sim \text{Ber}(\mu)$, if

$$\mathbf{P}(Y = 1) = \mu, \quad \mathbf{P}(Y = 0) = 1 - \mu,$$

or equivalently if its probability mass function (pmf) is $p(y; \mu) = \mu^y(1 - \mu)^{1-y}$ for $y \in \{0, 1\}$. Then $\mathbf{E} Y = \mu$ and $\text{var } Y = \mu(1 - \mu)$.

Definition 3.2 (Likelihood and maximum likelihood estimator). Given independent observations $y_1, \dots, y_n$ from a parametric family $\{p(y; \theta) : \theta \in \Theta\}$, the **likelihood** of θ is

$$L(\theta) := \prod_{i=1}^n p(y_i; \theta),$$

the **log-likelihood** is $\ell(\theta) := \log L(\theta) = \sum_{i=1}^n \log p(y_i; \theta)$, and the **maximum likelihood estimator** (MLE) is

$$\hat{\theta}_{\text{MLE}} := \arg \max_{\theta \in \Theta} \ell(\theta).$$

Example 3.1 (Bernoulli MLE is the sample proportion). Let $y_1, \dots, y_n \stackrel{\text{iid}}{\sim} \text{Ber}(\mu)$ and write $n_1 = \sum_i y_i$, $n_0 = n - n_1$. Then

$$\ell(\mu) = \sum_{i=1}^n [y_i \log \mu + (1 - y_i) \log(1 - \mu)] = n_1 \log \mu + n_0 \log(1 - \mu).$$

The first-order condition $\ell'(\mu) = n_1/\mu - n_0/(1 - \mu) = 0$ gives $\hat{\mu}_{\text{MLE}} = n_1/n$, the sample proportion of 1s. The second derivative $\ell''(\mu) = -n_1/\mu^2 - n_0/(1 - \mu)^2 < 0$ confirms strict concavity, so $\hat{\mu}$ is the unique maximizer.

The Bernoulli model is the simplest nontrivial likelihood. Logistic regression is Bernoulli MLE with μ allowed to depend on covariates.

Definition 3.3 (Kullback–Leibler divergence). For pmfs p, q on a finite set $\mathcal{Y}$ with $q(y) > 0$ wherever $p(y) > 0$, the **Kullback–Leibler divergence** from p to q is

$$D_{\text{KL}}(p \parallel q) := \sum_{y \in \mathcal{Y}} p(y) \log \frac{p(y)}{q(y)},$$

with the convention $0 \log 0 = 0$. $D_{\text{KL}}(p \parallel q)$ is not symmetric in (p, q) and is not a metric.

Lemma 3.4 (Gibbs's inequality). $D_{\text{KL}}(p \parallel q) \geq 0$, with equality iff $p = q$.

Proof. $\log$ is concave on $(0, \infty)$, so for any probability weights $w_y = p(y) \geq 0$ summing to 1 and any positive x_y ,

$$\sum_y p(y) \log x_y \leq \log \sum_y p(y) x_y. \quad (\text{Jensen, Unit 1 §1.9.9})$$

Apply this with $x_y = q(y)/p(y)$ on $\text{supp}(p) := \{y : p(y) > 0\}$:

$$-D_{\text{KL}}(p \parallel q) = \sum_{y \in \text{supp}(p)} p(y) \log \frac{q(y)}{p(y)} \leq \log \sum_{y \in \text{supp}(p)} q(y) \leq \log 1 = 0.$$

The last inequality is $\sum_{\text{supp}(p)} q \leq \sum_y q = 1$. Equality in Jensen's requires $q(y)/p(y)$ constant on $\text{supp}(p)$; combined with both sums equaling 1, this forces $p = q$. $\square$

Definition 3.5 (Cross-entropy). For pmfs p, q on $\mathcal{Y}$, the **cross-entropy** from p to q is

$$H(p, q) \; := \; - \sum_{y \in \mathcal{Y}} p(y) \log q(y),$$

and the (Shannon) **entropy** of p is $H(p) := H(p, p) = - \sum_y p(y) \log p(y)$. Algebraic manipulation gives

$$H(p, q) \; = \; H(p) + D_{\text{KL}}(p \parallel q).$$

Remark. Two consequences of $H(p, q) = H(p) + D_{\text{KL}}(p \parallel q)$ recur throughout this unit:

Use	Reason
With p fixed, $\arg \min_q H(p, q) = \arg \min_q D_{\text{KL}}(p \parallel q)$	$H(p)$ does not depend on q
$H(p, q) \geq H(p)$, equality iff $q = p$	Gibbs's inequality (3.4)

The first will identify cross-entropy minimization with maximum likelihood once the model is in hand.

1.2 The logistic regression model

Logistic regression assigns each observation a probability of being class 1 via a linear-then-squashed function of the covariates. The squashing is performed by the sigmoid.

Definition 3.6 (Sigmoid function). The **sigmoid** (or **logistic**) function $\sigma : \mathbb{R} \rightarrow (0, 1)$ is

$$\sigma(z) := \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}.$$

Lemma 3.7 (Sigmoid properties). σ is strictly increasing with $\sigma(\mathbb{R}) = (0, 1)$ and $\sigma(0) = 1/2$. As $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$; as $z \rightarrow +\infty$, $\sigma(z) \rightarrow 1$. Also,

$$1 - \sigma(z) = \sigma(-z), \quad \sigma'(z) = \sigma(z)(1 - \sigma(z)).$$

Proof. Symmetry: $1 - \sigma(z) = e^{-z}/(1 + e^{-z}) = 1/(1 + e^z) = \sigma(-z)$. Derivative, by the chain rule:

$$\sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z) \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z)).$$

Strict positivity of σ' gives strict monotonicity; the midpoint and limits are read off the formula. $\square$

Definition 3.8 (Logistic regression model). Given covariates $x_i \in \mathbb{R}^p$ (with $x_{i,1} \equiv 1$ when an intercept is included), the **logistic regression model** with parameter $\beta \in \mathbb{R}^p$ posits that $Y_1, \dots, Y_n$ are independent with

$$Y_i \sim \text{Ber}(\mu_i), \quad \mu_i = \sigma(x_i^\top \beta).$$

The quantity $\eta_i := x_i^\top \beta$ is the **linear predictor**.

The inverse of σ , the **logit**, is

$$\text{logit}(\mu) := \sigma^{-1}(\mu) = \log \frac{\mu}{1 - \mu},$$

so the model can be written equivalently as $\text{logit}(\mu_i) = x_i^\top \beta$. Coefficients β are therefore interpreted in log-odds: a unit increase in $x_{i,j}$ raises the log-odds of $Y_i = 1$ by β_j , equivalently multiplies the odds $\mu_i/(1 - \mu_i)$ by e^{β_j} .

Definition 3.9 (Plug-in classifier and decision boundary). Given threshold $\tau \in (0, 1)$, the **plug-in classifier** predicts

$$\hat{Y}(x) := \mathbf{1}[\sigma(x^\top \beta) > \tau].$$

By strict monotonicity of σ (3.7), this is equivalent to $x^\top \beta > \text{logit}(\tau)$, so the **decision boundary**

$$\mathcal{B}_\tau := \{x \in \mathbb{R}^p : x^\top \beta = \text{logit}(\tau)\}$$

is a hyperplane in $\mathbb{R}^p$. At $\tau = 1/2$, $\text{logit}(1/2) = 0$, so $\mathcal{B}_{1/2} = \{x : x^\top \beta = 0\}$.

The boundary is a hyperplane: class 1 on one side, class 0 on the other. The sigmoid's curvature shows in the predicted probabilities, not in the geometry of the classifier.

1.3 Cross-entropy loss and MLE

For logistic regression (3.8), the log-likelihood is

$$\ell(\beta) = \sum_{i=1}^n [y_i \log \mu_i + (1 - y_i) \log(1 - \mu_i)], \quad \mu_i = \sigma(x_i^\top \beta).$$

Definition 3.10 (Binary cross-entropy loss). The **binary cross-entropy loss** for label $y \in \{0, 1\}$ and predicted probability $\mu \in (0, 1)$ is

$$L_{\text{CE}}(y, \mu) := -y \log \mu - (1 - y) \log(1 - \mu).$$

Also called **log-loss** in machine-learning libraries (`sklearn.metrics.log_loss`, `xgboost`'s `logloss` objective).

Lemma 3.11 (MLE equals empirical cross-entropy minimization).

$$\hat{\beta}_{\text{MLE}} = \arg \min_{\beta \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n L_{\text{CE}}(y_i, \sigma(x_i^\top \beta)).$$

Proof. $\hat{\beta}_{\text{MLE}} = \arg \max_{\beta} \ell(\beta) = \arg \min_{\beta} -\ell(\beta)$, and $-\ell(\beta) = \sum_i L_{\text{CE}}(y_i, \mu_i)$ by inspection; the $1/n$ factor does not change the argmin. $\square$

Information-theoretic reading. Define the empirical pmf $\hat{p}_i$ on $\{0, 1\}$ by $\hat{p}_i(1) = y_i$, $\hat{p}_i(0) = 1 - y_i$ (a point mass at y_i), and the model pmf q_i by $q_i(1) = \mu_i$, $q_i(0) = 1 - \mu_i$. Direct calculation

gives

$$L_{\text{CE}}(y_i, \mu_i) = H(\hat{p}_i, q_i) = D_{\text{KL}}(\hat{p}_i \parallel q_i),$$

where the second equality uses $H(\hat{p}_i) = 0$ (entropy of a point mass) in the decomposition $H(p, q) = H(p) + D_{\text{KL}}(p \parallel q)$ (3.5). Hence

$$\hat{\beta}_{\text{MLE}} = \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n D_{\text{KL}}(\hat{p}_i \parallel q_i) :$$

MLE minimizes the average per-observation KL from labels to model.

1.4 Why not squared loss

Cross-entropy is the likelihood loss (3.11). The squared loss

$$L_{\text{SQ}}(y, \mu) := (y - \mu)^2$$

familiar from Unit 2 is the obvious alternative. The empirical risk under L_{CE} is convex in β ; under L_{SQ} it is not.

Lemma 3.12 (Cross-entropy is convex in β). $\beta \mapsto \sum_{i=1}^n L_{\text{CE}}(y_i, \sigma(x_i^\top \beta))$ is convex on $\mathbb{R}^p$.

Proof. Define the **softplus** $s(z) := \log(1 + e^z)$. Then $s'(z) = \sigma(z)$ and $s''(z) = \sigma(z)(1 - \sigma(z)) \geq 0$ (3.7), so s is convex by the second-order condition (Unit 1, §1.9.6). Algebra gives $-\log \sigma(\eta) = s(-\eta)$ and $-\log(1 - \sigma(\eta)) = s(\eta)$, hence

$$L_{\text{CE}}(y, \sigma(\eta)) = y s(-\eta) + (1 - y) s(\eta).$$

Both $s(-\eta)$ and $s(\eta)$ are convex in η (composition of s with an affine map; Unit 1, §1.9.7), and $y, 1 - y \geq 0$, so the sum is convex. Composing further with $\eta = x_i^\top \beta$ (affine in β) and summing over i preserves convexity. $\square$

Lemma 3.13 (Squared loss is not convex in β). For any $x \neq 0$ and $y \in \{0, 1\}$, $\beta \mapsto L_{\text{SQ}}(y, \sigma(x^\top \beta))$ is not convex on $\mathbb{R}^p$.

Proof. For $y = 1$, set $\phi(\eta) := (1 - \sigma(\eta))^2$. With $\mu := \sigma(\eta)$ and $\sigma'(\eta) = \mu(1 - \mu)$,

$$\phi'(\eta) = -2\mu(1 - \mu)^2, \quad \phi''(\eta) = -2\mu(1 - \mu)^2(1 - 3\mu),$$

so $\phi''(\eta) < 0$ when $\mu < 1/3$. Along the line $\beta_0 + tx$, $\beta \mapsto L_{\text{SQ}}(1, \sigma(x^\top \beta))$ becomes $g(t) = \phi(x^\top \beta_0 + t\|x\|^2)$ with $g''(t) = \|x\|^4 \phi''(\cdot)$, also negative there; convexity therefore fails (Unit 1, §1.9.4). The case $y = 0$ is symmetric: $\phi'' < 0$ when $\mu > 2/3$. $\square$

Remark. Confidently wrong predictions ($y = 1, \mu < 1/3$ or $y = 0, \mu > 2/3$) are where squared loss fails twice: the loss is concave (just proved), and its gradient $\partial L_{\text{SQ}}/\partial \eta = -2(y - \mu)\mu(1 - \mu)$ vanishes as $\mu \rightarrow 0$ or 1. Cross-entropy fails neither: $\partial L_{\text{CE}}/\partial \eta = \mu - y$ stays $O(1)$, and the loss is globally convex (3.12).

1.5 Optimization: gradient and IRLS

The first-order condition $\nabla \ell(\beta) = 0$ is nonlinear in β (unlike OLS), so logistic regression has no closed-form MLE. Newton-Raphson iterates have a weighted-least-squares interpretation that runs underneath `glm`.

Lemma 3.14 (Gradient and Hessian). Let $X \in \mathbb{R}^{n \times p}$ have rows $x_i^\top$, $y \in \mathbb{R}^n$ be the response, $\mu(\beta) \in (0, 1)^n$ the predicted means with $\mu_i = \sigma(x_i^\top \beta)$, and $W(\beta) := \text{diag}(\mu_i(1 - \mu_i)) \in \mathbb{R}^{n \times n}$.

Then

$$\nabla \ell(\beta) = X^\top (y - \mu(\beta)), \quad \nabla^2 \ell(\beta) = -X^\top W(\beta) X.$$

Proof. By the chain rule and $\sigma'(\eta) = \mu(1 - \mu)$, $\partial \mu_i / \partial \beta_j = \mu_i(1 - \mu_i) x_{ij}$. Hence

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^n \left[\frac{y_i}{\mu_i} - \frac{1 - y_i}{1 - \mu_i} \right] \mu_i(1 - \mu_i) x_{ij} = \sum_{i=1}^n (y_i - \mu_i) x_{ij} = (X^\top (y - \mu))_j,$$

and differentiating once more,

$$\frac{\partial^2 \ell}{\partial \beta_j \partial \beta_k} = - \sum_{i=1}^n x_{ij} \mu_i(1 - \mu_i) x_{ik} = -(X^\top W X)_{jk}.$$

□

Remark. The diagonal entries of $W(\beta) = \text{diag}(\mu_i(1 - \mu_i))$ are non-negative (since $\mu_i \in (0, 1)$), so $W \succcurlyeq 0$. For any $v \in \mathbb{R}^p$,

$$v^\top X^\top W X v = (Xv)^\top W (Xv) \geq 0,$$

hence $X^\top W X \succcurlyeq 0$ and $-\ell$ has PSD Hessian: a second proof of 3.12.

Theorem 3.15 (Iteratively reweighted least squares). With $\mu^{(k)} := \mu(\beta^{(k)})$ and $W^{(k)} := W(\beta^{(k)})$, the Newton-Raphson step on ℓ at $\beta^{(k)}$ is

$$\beta^{(k+1)} = (X^\top W^{(k)} X)^{-1} X^\top W^{(k)} z^{(k)}, \quad z^{(k)} := X\beta^{(k)} + (W^{(k)})^{-1}(y - \mu^{(k)}).$$

The right-hand side is the weighted least squares estimate $\arg \min_{\beta} \|W^{(k)1/2}(z^{(k)} - X\beta)\|^2$.

Proof. Newton on ℓ : $\beta^{(k+1)} = \beta^{(k)} - (\nabla^2 \ell)^{-1} \nabla \ell$. By 3.14,

$$\beta^{(k+1)} = \beta^{(k)} + (X^\top W^{(k)} X)^{-1} X^\top (y - \mu^{(k)}).$$

Write $\beta^{(k)} = (X^\top W^{(k)} X)^{-1} (X^\top W^{(k)} X) \beta^{(k)}$ and $X^\top (y - \mu^{(k)}) = X^\top W^{(k)} (W^{(k)})^{-1} (y - \mu^{(k)})$,

then combine:

$$\beta^{(k+1)} = (X^\top W^{(k)} X)^{-1} X^\top W^{(k)} [X\beta^{(k)} + (W^{(k)})^{-1}(y - \mu^{(k)})] = (X^\top W^{(k)} X)^{-1} X^\top W^{(k)} z^{(k)}.$$

This is the solution of the WLS normal equations $X^\top W^{(k)} X\beta = X^\top W^{(k)} z^{(k)}$. □

Remark. 3.15 is the algorithm `glm(family = binomial)` runs: each iteration recomputes $W^{(k)}$ and $z^{(k)}$, then solves one WLS. Convergence is quadratic near $\hat{\beta}_{\text{MLE}}$, where $X^\top W X$ is invertible.

2 Discrete Evaluation Metrics

2.1 The confusion matrix

Each observation has a true label $y_i \in \{0, 1\}$ and a predicted label $\hat{y}_i \in \{0, 1\}$ (for logistic regression, $\hat{y}_i = \mathbb{1}[\sigma(x_i^\top \beta) > \tau]$ via 3.9). Unit 1 §1.8.2 introduced the four counts; we restate them in indicator form, add marginal totals, and anchor the derivations that follow.

Definition 3.16 (Confusion matrix). For $\{(y_i, \hat{y}_i)\}_{i=1}^n$, the four **confusion counts** are

$$\text{TP} := \sum_i \mathbb{1}[y_i = 1, \hat{y}_i = 1], \quad \text{FN} := \sum_i \mathbb{1}[y_i = 1, \hat{y}_i = 0],$$

$$\text{FP} := \sum_i \mathbb{1}[y_i = 0, \hat{y}_i = 1], \quad \text{TN} := \sum_i \mathbb{1}[y_i = 0, \hat{y}_i = 0],$$

arranged as the **confusion matrix**:

	Predicted 1	Predicted 0
Actual 1	TP	FN
Actual 0	FP	TN

The **marginal counts** are $P := \text{TP} + \text{FN}$ (actual positives), $N := \text{FP} + \text{TN}$ (actual negatives), $\hat{P} := \text{TP} + \text{FP}$ (predicted positives), and $\hat{N} := \text{FN} + \text{TN}$ (predicted negatives), with $P + N = \hat{P} + \hat{N} = n$.

Remark. The first letter (T/F) records whether the prediction matched y ; the second (P/N) records the value of $\hat{y}$. Diagonal entries TP and TN are correct; off-diagonal FP and FN are errors.

Definition 3.17 (Empirical operators). Hatted operators are sample analogues of their population versions. For the sample $\{(y_i, \hat{y}_i)\}_{i=1}^n$, with A, B predicates and Z, Z_1, Z_2 real-valued functions of $(y, \hat{y})$,

$$\hat{P}(A) := \frac{1}{n} \sum_i \mathbb{1}[A(y_i, \hat{y}_i)], \quad \hat{P}(A \mid B) := \frac{\hat{P}(A \wedge B)}{\hat{P}(B)} \quad (\text{when } \hat{P}(B) > 0),$$

$$\hat{E}(Z) := \frac{1}{n} \sum_i Z(y_i, \hat{y}_i), \quad \widehat{\text{var}}(Z) := \hat{E}(Z^2) - \hat{E}(Z)^2, \quad \widehat{\text{cov}}(Z_1, Z_2) := \hat{E}(Z_1 Z_2) - \hat{E}(Z_1) \hat{E}(Z_2).$$

For example, $\hat{P}(Y = 1, \hat{Y} = 1) = \text{TP}/n$, $\hat{P}(Y = 1) = P/n$, and $\hat{P}(\hat{Y} = 1 \mid Y = 1) = \text{TP}/P$.

2.2 Precision, recall, and F1

Precision and recall are conditional fractions read off the confusion matrix (3.16); F_1 combines them into a single score.

Definition 3.18 (Precision, recall, specificity). For a binary classifier on the test set,

Metric	Formula	Interpretation
Precision	$\frac{TP}{TP + FP} = \frac{TP}{\hat{P}}$	$\hat{P}(Y = 1 \mid \hat{Y} = 1)$
Recall (TPR, sensitivity)	$\frac{TP}{TP + FN} = \frac{P}{\hat{P}}$	$\hat{P}(\hat{Y} = 1 \mid Y = 1)$
Specificity (TNR)	$\frac{TN}{TN + FP} = \frac{TN}{N}$	$\hat{P}(\hat{Y} = 0 \mid Y = 0)$

The **false positive rate** is $FPR := FP/N = 1 - \text{Specificity}$.

Remark. Precision and recall share a key trait: both ignore TN, so only the positive class enters either formula. Appropriate when positives are a rare signal against an abundant negative background (fraud, disease, anomaly); unhelpful when both classes matter symmetrically.

- **Precision** is conditioned on the prediction: it tracks the cost of false alarms.
- **Recall** is conditioned on the ground truth: it tracks the cost of missed targets.

Definition 3.19 (F_β score). For $\beta > 0$ and a classifier with precision $p > 0$ and recall $r > 0$, the F_β **score** is

$$F_\beta := (1 + \beta^2) \cdot \frac{p \cdot r}{\beta^2 p + r}.$$

The F_1 **score** is $F_1 = \frac{2pr}{p+r}$, the **harmonic mean** of p and r : $F_1^{-1} = \frac{1}{2}(p^{-1} + r^{-1})$. By convention $F_\beta = 0$ when $p = 0$ or $r = 0$.

Remark. Harmonic mean penalizes asymmetry: $F_1 \leq 2 \min(p, r)$, so $F_1 \rightarrow 0$ if either $p \rightarrow 0$ or $r \rightarrow 0$. Arithmetic mean $(p + r)/2$ averages: a high p masks a low r . The β in F_β tunes the weight: $\beta > 1$ favors recall (F_2 when missing a positive costs more than a false alarm), $\beta < 1$ favors precision.

2.3 Accuracy, balanced accuracy, and MCC

Three single-number summaries of the confusion matrix.

Definition 3.20 (Accuracy and balanced accuracy).

$$\text{Acc} := \frac{\text{TP} + \text{TN}}{n}, \quad \text{BA} := \frac{1}{2}(\text{TPR} + \text{TNR}) = \frac{1}{2} \left(\frac{\text{TP}}{P} + \frac{\text{TN}}{N} \right).$$

Remark. On Unit 1 §1.8.3's 99/1 imbalanced dataset, the trivial $\hat{y} \equiv 0$ has $\text{Acc} = 0.99$ but $\text{TPR} = 0$. Balanced accuracy corrects this: $\text{BA} = (0 + 1)/2 = 0.5$, the random-baseline value. $\text{BA} = \text{Acc}$ when $P = N$.

Definition 3.21 (Matthews correlation coefficient). Provided $P, N, \hat{P}, \hat{N} > 0$,

$$\text{MCC} := \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{\sqrt{P \cdot N \cdot \hat{P} \cdot \hat{N}}}.$$

Lemma 3.22 (MCC = sample correlation). MCC is the sample Pearson correlation between $\{y_i\}$ and $\{\hat{y}_i\}$, hence lies in $[-1, +1]$: $+1$ iff $\hat{y}_i = y_i$ for all i , -1 iff $\hat{y}_i = 1 - y_i$ for all i , 0 iff $\text{TP} \cdot \text{TN} = \text{FP} \cdot \text{FN}$.

Proof. Since $y_i, \hat{y}_i \in \{0, 1\}$, $y_i^2 = y_i$, so $\widehat{\text{var}}(Y) = \frac{P}{n} - \left(\frac{P}{n}\right)^2 = \frac{PN}{n^2}$ and likewise $\widehat{\text{var}}(\hat{Y}) = \frac{\hat{P}\hat{N}}{n^2}$. The sample covariance is

$$\widehat{\text{cov}}(Y, \hat{Y}) = \frac{\text{TP}}{n} - \frac{P}{n} \cdot \frac{\hat{P}}{n} = \frac{n\text{TP} - P\hat{P}}{n^2} = \frac{\text{TP} \cdot \text{TN} - \text{FP} \cdot \text{FN}}{n^2},$$

the last equality by expanding $n = \text{TP} + \text{FP} + \text{TN} + \text{FN}$, $P = \text{TP} + \text{FN}$, $\hat{P} = \text{TP} + \text{FP}$. Then $\widehat{\text{cov}}(Y, \hat{Y}) / \sqrt{\widehat{\text{var}}(Y)\widehat{\text{var}}(\hat{Y})}$ reduces to the MCC formula. The bound $|\text{Corr}| \leq 1$ is Cauchy-Schwarz, with equality at exact agreement or anti-agreement. $\square$

Remark. Acc is fooled by imbalance (Unit 1 §1.8.3); BA and MCC are insensitive to the prevalence. F_1 (§2.2) focuses on positives. Choose by context: Acc for balanced classes; BA or MCC for general single-number use; F_1 when only the positive class matters.

2.4 Cost matrices and weighted error

Most standard metrics embed a rigid, predefined trade-off: Acc weights all errors equally, BA and MCC force a balance between the two classes, and F_1 completely ignores TN. Cost matrices invert

this paradigm: instead of adopting the metric's implicit assumptions, the analyst explicitly specifies what each error type costs the business.

Definition 3.23 (Cost matrix and total cost). A **cost matrix** is a 2×2 matrix C whose entry $C_{y,\hat{y}}$ is the cost of predicting $\hat{y}$ when the actual class is y . The standard choice

$$C_{00} = C_{11} = 0, \quad C_{01} = c_{\text{FP}}, \quad C_{10} = c_{\text{FN}},$$

charges nothing for correct predictions. The **total cost** on the test set is

$$\text{TotalCost} := \sum_{i=1}^n C_{y_i, \hat{y}_i} = c_{\text{FP}} \cdot \text{FP} + c_{\text{FN}} \cdot \text{FN}.$$

Remark. The trade-off is the ratio $c_{\text{FN}}/c_{\text{FP}}$, not the absolute costs:

Setting	$c_{\text{FN}}/c_{\text{FP}}$	Reading
Spam filter (precision priority)	$\ll 1$	a missed spam is cheap; a falsely flagged ham is costly
Fraud detection (recall priority)	$\gg 1$	a missed fraud loses money; a flagged clean transaction costs review
Disease screening (recall priority)	$\gg 1$	a missed cancer can be fatal; a follow-up scan is recoverable
Symmetric (textbook accuracy)	$= 1$	total cost reduces to $\text{FP} + \text{FN} = n(1 - \text{Acc})$

The dataset and the metric do not pick the ratio; the business context does.

Lemma 3.24 (Acc and BA as cost-weighted errors). Under the standard cost matrix:

- $1 - \text{Acc} = \text{TotalCost}/n$ when $c_{\text{FP}} = c_{\text{FN}}$.
- $1 - \text{BA} = \text{TotalCost}$ when $c_{\text{FP}} = 1/(2N)$ and $c_{\text{FN}} = 1/(2P)$.

Accuracy thus encodes equal per-error cost; balanced accuracy encodes inverse-class-size weighting.

Proof. For Acc: $\sum_i \mathbf{1}[y_i \neq \hat{y}_i] = \text{FP} + \text{FN}$, so $1 - \text{Acc} = (\text{FP} + \text{FN})/n$. Setting $c_{\text{FP}} = c_{\text{FN}} = 1$ gives $\text{TotalCost} = \text{FP} + \text{FN}$, and dividing by n matches.

For BA: $\text{BA} = \frac{1}{2}(\text{TP}/P + \text{TN}/N) = 1 - \frac{1}{2}(\text{FN}/P + \text{FP}/N)$. Setting $c_{\text{FP}} = 1/(2N)$, $c_{\text{FN}} = 1/(2P)$

gives $\text{TotalCost} = \text{FP}/(2N) + \text{FN}/(2P) = 1 - \text{BA}$. □

Remark. Cost-weighted error generalizes Acc and BA (3.24); the framework exposes the trade-off as a parameter the analyst chooses, rather than embedding it in the metric definition.

3 ROC and PR Curves

3.1 From scores to predictions: the threshold

Until §2, metrics were functions of $\hat{y}_i \in \{0, 1\}$. The classifier itself produces a continuous **score**; thresholding yields $\hat{y}_i$. The threshold trades precision against recall (equivalently, FP against FN); ROC and PR curves visualize this trade-off across all thresholds.

Definition 3.25 (Scoring classifier and threshold-indexed counts). A **scoring classifier** produces $s_i \in \mathbb{R}$ for each observation, with larger s favoring class 1. For threshold $\tau \in \mathbb{R}$, the prediction is $\hat{y}_i(\tau) := \mathbb{1}[s_i > \tau]$, and the confusion counts at τ are

$$\begin{aligned} \text{TP}(\tau) &:= \sum_i \mathbb{1}[y_i = 1, s_i > \tau], & \text{FN}(\tau) &:= \sum_i \mathbb{1}[y_i = 1, s_i \leq \tau], \\ \text{FP}(\tau) &:= \sum_i \mathbb{1}[y_i = 0, s_i > \tau], & \text{TN}(\tau) &:= \sum_i \mathbb{1}[y_i = 0, s_i \leq \tau]. \end{aligned}$$

Every metric of §2 inherits a τ -dependence: $\text{Recall}(\tau)$, $\text{Precision}(\tau)$, $\text{Acc}(\tau)$, $\dots$

Remark. For logistic regression, s_i may be either the linear predictor $x_i^\top \beta$ or the probability $\mu_i = \sigma(x_i^\top \beta)$. σ is strictly increasing (3.7), so the two scales are equivalent for thresholding: $\sigma(x_i^\top \beta) > \tau' \iff x_i^\top \beta > \text{logit}(\tau')$. The probability scale is conventional for plotting.

Lemma 3.26 (Boundary behavior). $\text{TP}(\tau)$ and $\text{FP}(\tau)$ are non-increasing in τ (raising the threshold predicts fewer positives), with $\text{TP}(\tau) = \text{FP}(\tau) = 0$ for $\tau > \max_i s_i$ and $\text{TP}(\tau) = P$, $\text{FP}(\tau) = N$ for $\tau < \min_i s_i$.

Proof. $\mathbb{1}[s_i > \tau]$ is non-increasing in τ , so each sum is too. For $\tau > \max_i s_i$, every indicator vanishes. For $\tau < \min_i s_i$, every indicator equals 1, so TP counts $\{i : y_i = 1\} = P$ and FP counts $\{i : y_i = 0\} = N$. $\square$

3.2 ROC curve and AUC

The **ROC curve** (Receiver Operating Characteristic) plots how the false positive rate trades against the true positive rate as the threshold sweeps. Its area, AUC, summarizes ranking quality independent of any threshold.

Definition 3.27 (ROC curve). For a scoring classifier (3.25), the **ROC curve** is

$$\text{ROC} := \{(\text{FPR}(\tau), \text{TPR}(\tau)) : \tau \in \mathbb{R}\} \subseteq [0, 1]^2,$$

with $\text{TPR}(\tau) = \text{TP}(\tau)/P$ and $\text{FPR}(\tau) = \text{FP}(\tau)/N$.

Remark. The curve runs from $(0, 0)$ at $\tau \rightarrow +\infty$ to $(1, 1)$ at $\tau \rightarrow -\infty$ (3.26). Both coordinates are non-increasing in τ , so the curve moves up and to the right as τ decreases. On n test points it is a step function with at most $n + 1$ vertices: jumps occur at distinct score values.

Definition 3.28 (Area under the ROC curve). The **area under the ROC curve** (AUC) is

$$\text{AUC} := \int_0^1 \text{TPR}(\text{FPR}^{-1}(t)) \, dt,$$

with $\text{FPR}^{-1}(t) := \inf\{\tau : \text{FPR}(\tau) \leq t\}$ (generalized inverse, since FPR is a step function).

Lemma 3.29 (AUC equals the empirical Mann–Whitney statistic). With ties broken by averaging,

$$\text{AUC} = \frac{1}{P \cdot N} \sum_{\substack{i: y_i=1 \\ j: y_j=0}} \left(\mathbb{1}[s_i > s_j] + \frac{1}{2} \mathbb{1}[s_i = s_j] \right).$$

Equivalently, AUC is the empirical probability that a uniformly random positive scores higher than a uniformly random negative (half-credit on ties).

Proof. Sort the negatives' scores in descending order $s_{j(1)} > \cdots > s_{j(N)}$ (distinct case; ties below). As τ decreases past $s_{j(k)}$, FPR jumps from $(k-1)/N$ to k/N , with TPR on $((k-1)/N, k/N)$ equal to $\frac{1}{P} \# \{i : y_i = 1, s_i > s_{j(k)}\}$. Hence

$$\text{AUC} = \sum_{k=1}^N \frac{1}{N} \cdot \frac{\# \{i : y_i = 1, s_i > s_{j(k)}\}}{P} = \frac{1}{PN} \sum_{i: y_i=1} \sum_{j: y_j=0} \mathbb{1}[s_i > s_j].$$

A tied pair (s_i, s_j) produces a diagonal segment in the curve, contributing a triangle of area $\frac{1}{2} \cdot \frac{1}{N} \cdot \frac{1}{P}$, the $\frac{1}{2}$ factor. $\square$

Remark. Boundary cases and invariance:

Classifier	AUC	ROC geometry
Perfect ranking ($s_i > s_j$ for all $y_i = 1, y_j = 0$)	1	passes through $(0, 1)$
Random scores (independent of y)	$\frac{1}{2}$	diagonal $\text{TPR} = \text{FPR}$
Inverted ranking ($s_i < s_j$ for all $y_i = 1, y_j = 0$)	0	passes through $(1, 0)$

AUC depends only on the ranking of $\{s_i\}$: any strictly increasing transformation g leaves it unchanged, since $g(s_i) > g(s_j) \iff s_i > s_j$. In particular, for logistic regression AUC is the same on the linear-predictor and probability scales.

3.3 PR curve and average precision

The PR curve plots precision against recall as the threshold sweeps; its area, average precision (AP), summarizes ranking quality with a different emphasis from AUC.

Definition 3.30 (PR curve). For a scoring classifier (3.25), the **precision–recall (PR) curve**

is

$$\text{PR} := \left\{ (\text{Recall}(\tau), \text{Precision}(\tau)) : \tau \in \mathbb{R}, \text{TP}(\tau) + \text{FP}(\tau) > 0 \right\}.$$

Remark. At $\tau \rightarrow -\infty$, $\text{Recall} = 1$ and $\text{Precision} = P/n$ (the prevalence). At τ above $\max_i s_i$, no positive is predicted, so Precision is $0/0$; by convention the curve is anchored at $(0, 1)$ for plotting. Like ROC, PR is a step function with jumps at distinct score values.

Definition 3.31 (Average precision). **Average precision** (AP) is the Riemann sum

$$\text{AP} := \sum_k (\text{Recall}(\tau_k) - \text{Recall}(\tau_{k-1})) \cdot \text{Precision}(\tau_k),$$

summing over thresholds τ_k at which Recall jumps (i.e., at distinct positive-class score values).

Lemma 3.32 (AP as the mean of precision over positives). If scores are distinct,

$$\text{AP} = \frac{1}{P} \sum_{i: y_i=1} \text{Precision}(s_i^-),$$

where $\text{Precision}(s_i^-)$ is the precision at the threshold just below s_i (when positive i enters the predicted-positive set).

Proof. Index the recall-jump thresholds by the P positives in decreasing score order: $\tau_k^* = s_{i(k)}^-$ for the k -th positive. Each jump moves Recall by exactly $1/P$ (one new TP, P unchanged), so the k -th step contributes $\text{Precision}(\tau_k^*)/P$. Summing $k = 1, \dots, P$ gives the formula. $\square$

Remark. Boundary cases:

Classifier	AP	Reading
Perfect ranking ($s_i > s_j$ for all $y_i = 1, y_j = 0$)	1	every $\text{Precision}(s_i^-) = 1$
Random scores (independent of y)	$\mathbb{E}[\text{AP}] = P/n$	by uniform-permutation symmetry

The random baseline for AP is the prevalence P/n , not $1/2$ as for AUC. AP's baseline tracks the imbalance, so AP is a sharper diagnostic than AUC when positives are rare.

Remark (Computation: ties and interpolation).

- **Tied scores.** When observations share a score s , recall jumps by k_+/P (with k_+ positives at s), and precision uses the cumulative TP, FP through the tied block. Tree-based models often produce such ties; `yardstick::pr_auc` and `roc_auc` treat tied groups as a single jump. 3.32 generalizes: average Precision(s_i^-) within each tied block.
- **Linear interpolation in PR space is wrong.** A line segment between (R_{k-1}, P_{k-1}) and (R_k, P_k) over-estimates AP: precision $TP/(TP + FP)$ traces a hyperbola between the two points, not a chord. Use the step-sum form of 3.31 directly; for plotting, draw steps, not diagonal connectors.

3.4 ROC vs PR under imbalance

When $P \ll N$ (positives are rare), ROC and PR diagnose the same classifier differently. ROC normalizes FP by N , so a large absolute FP count can register as a small FPR; PR puts FP in the denominator of precision, where it competes directly with TP.

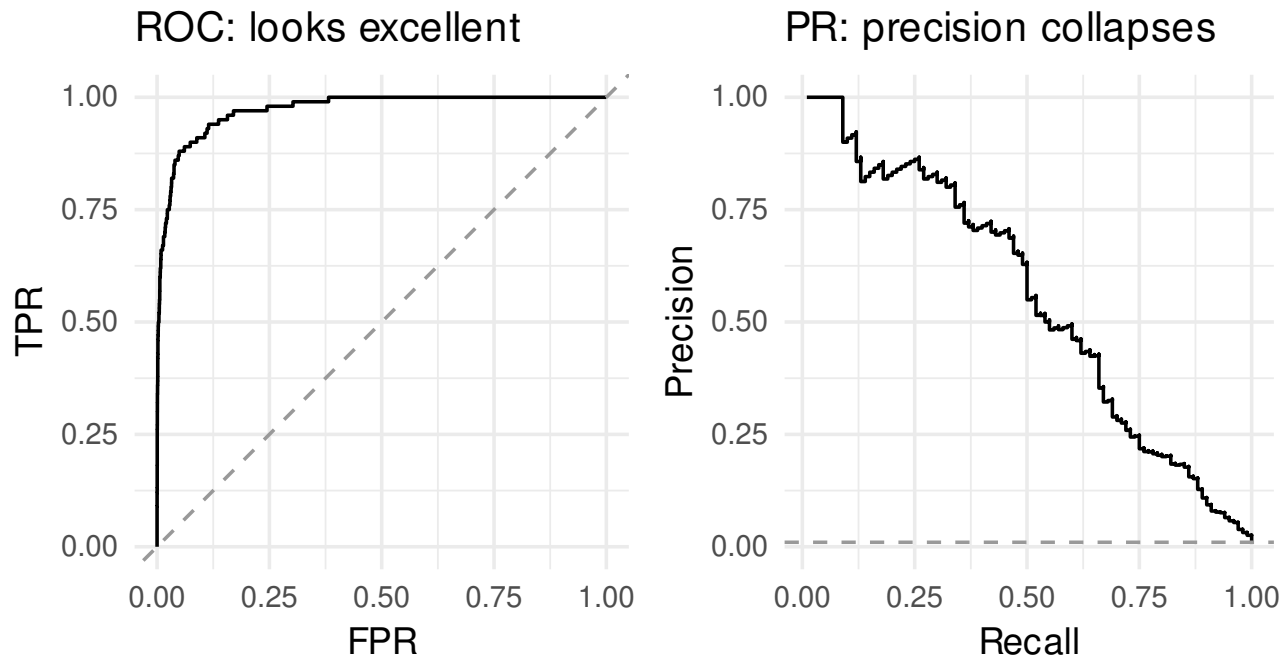
Lemma 3.33 (Precision in terms of TPR and FPR). For any threshold τ with $\text{TP}(\tau) + \text{FP}(\tau) > 0$,

$$\text{Precision}(\tau) = \frac{\text{TPR}(\tau)}{\text{TPR}(\tau) + \frac{N}{P} \text{FPR}(\tau)}.$$

Proof. Precision = $\text{TP}/(\text{TP} + \text{FP})$. Divide numerator and denominator by P :

$$\text{Precision} = \frac{\text{TP}/P}{\text{TP}/P + \text{FP}/P}.$$

$\text{TP}/P = \text{TPR}$ by definition; $\text{FP}/P = (N/P)(\text{FP}/N) = (N/P)\text{FPR}$. Substituting gives the formula. $\square$



Same classifier, $N/P = 99$ imbalance: ROC hugs the top-left ($\text{AUC} \approx 0.97$), but precision drops with recall, approaching the prevalence baseline $P/(P + N) = 0.01$ (dashed) at high recall.

Remark. The factor N/P amplifies FPR’s contribution to the denominator of precision. On a 1:100-imbalanced dataset ($N/P = 100$), a classifier with $\text{TPR} = 1$ and $\text{FPR} = 0.02$ has

$$\text{Precision} = \frac{1}{1 + 100 \cdot 0.02} = \frac{1}{3},$$

so two-thirds of predicted positives are wrong despite an apparently small FPR. ROC marks this point near the upper-left corner; PR marks it as low precision at $\text{Recall} = 1$. The diagnostic choice between curves therefore depends on the imbalance:

Setting	Preferred curve / metric
Balanced classes ($P \approx N$)	ROC / AUC; PR / AP carries the same information
Mild imbalance ($N/P \lesssim 5$)	either; report both
Severe imbalance ($N/P \gg 1$; fraud, rare disease)	PR / AP; AUC over-optimistic
Both classes equally costly	ROC / AUC
Positive class drives the decision (recall priority)	PR / AP

The choice is not aesthetic: report the curve whose axes track the business cost.

4 Threshold Selection

4.1 Bayes-optimal threshold under a cost matrix

Sections 3.1–3.4 traced curves over all τ ; deployment forces one. The Bayes-optimal τ minimizes expected cost under the matrix of §2.4.

Setup. Let $\mu(x) := \mathbf{P}(Y = 1 \mid X = x)$. Assume the score is **calibrated**: $s(x) = \mu(x)$ for all x . The prediction rule $\hat{y}(x; \tau) := \mathbf{1}\{s(x) \geq \tau\}$ then equals $\mathbf{1}\{\mu(x) \geq \tau\}$. Allow nonzero correct-decision costs c_{00}, c_{11} in the cost matrix of §2.4, alongside $c_{01} = c_{\text{FP}}$ and $c_{10} = c_{\text{FN}}$. The conditional expected cost at x given $\hat{y} \in \{0, 1\}$ is

$$R(\hat{y} \mid x) = c_{1,\hat{y}} \mu(x) + c_{0,\hat{y}} (1 - \mu(x)).$$

Theorem 3.34 (Bayes-optimal threshold). If $c_{10} > c_{11}$ and $c_{01} > c_{00}$ (mistakes strictly costlier than correct decisions on the same row), the minimizer of $R(\hat{y} \mid x)$ is $\hat{y}(x) = \mathbf{1}\{\mu(x) \geq \tau^*\}$ with

$$\tau^* = \frac{c_{01} - c_{00}}{(c_{01} - c_{00}) + (c_{10} - c_{11})}.$$

Proof. Predict $\hat{y} = 1$ iff $R(1 \mid x) \leq R(0 \mid x)$:

$$c_{11} \mu(x) + c_{01} (1 - \mu(x)) \leq c_{10} \mu(x) + c_{00} (1 - \mu(x)).$$

Collect $\mu(x)$ on the left:

$$(c_{10} - c_{11}) \mu(x) \geq (c_{01} - c_{00}) (1 - \mu(x)).$$

Both bracketed differences are positive by hypothesis. Solving for $\mu(x)$,

$$\mu(x) [(c_{10} - c_{11}) + (c_{01} - c_{00})] \geq c_{01} - c_{00},$$

yields $\mu(x) \geq \tau^*$. □

Remark.

- **Cost-ratio simplification.** With correct decisions free ($c_{00} = c_{11} = 0$, as in §2.4), $\tau^* = c_{\text{FP}} / (c_{\text{FP}} + c_{\text{FN}})$ depends only on the FP:FN ratio. If FN is $9\times$ as costly as FP (missed disease vs. false alarm), $\tau^* = 1/10$: predict $\hat{y} = 1$ whenever $\mu(x) \geq 0.1$. The default $\tau = 0.5$ is Bayes-optimal only when $c_{\text{FP}} = c_{\text{FN}}$.
- **Calibration is the central assumption.** The logistic MLE fits s within the parametric class $\sigma(x^\top \beta)$; a misspecified model has $s \neq \mu$. With $s = \mu - \delta$, thresholding s at $\tau^* = 0.1$ flags positives only at $\mu \geq 0.1 + \delta$, missing the rare positives the cost ratio was meant to catch. Empirical threshold tuning sidesteps the assumption; post-hoc recalibration restores it.

4.2 Empirical threshold from validation data

Without calibration, τ^* of 3.34 is unreachable: it is computed from μ but applied to s . The remedy: pick the threshold directly on a held-out sample by minimizing observed cost, with no assumption on s .

Setup. Let $\mathcal{D}_{\text{val}} = \{(x_i, y_i)\}_{i=1}^{n_v}$ be labeled and disjoint from training, with model scores $s_i := s(x_i)$.

Definition 3.35 (Empirical cost and tuned threshold). Under the standard cost matrix ($c_{00} = c_{11} = 0$) of §2.4,

$$\widehat{\text{Cost}}(\tau) := \sum_{i=1}^{n_v} C_{y_i, \hat{y}_i(\tau)} = c_{\text{FP}} \cdot \text{FP}(\tau) + c_{\text{FN}} \cdot \text{FN}(\tau),$$

where $\text{FP}(\tau), \text{FN}(\tau)$ are the threshold-indexed counts on $\mathcal{D}_{\text{val}}$ (3.25). The **empirical optimal threshold** is

$$\hat{\tau} := \arg \min_{\tau \in \mathbb{R}} \widehat{\text{Cost}}(\tau).$$

Remark.

- **Computation.** $\widehat{\text{Cost}}(\tau)$ is piecewise constant, jumping only at $\tau \in \{s_1, \dots, s_{n_v}\}$. Sort once and sweep, updating FP, FN by ± 1 at each crossing: $O(n_v \log n_v)$. Geometrically, $\hat{\tau}$ is the ROC point (3.27) furthest in direction $(-c_{\text{FP}}N, c_{\text{FN}}P)$ from the origin.
- **Variance and reuse.** $\hat{\tau}$ inherits the variance of $\mathcal{D}_{\text{val}}$: at $n_v = 200$ with 1% positive rate, only 2 positives exist, and a single mislabeled point shifts $\hat{\tau}$ across many score levels. Cross-validated tuning (Unit 4) reduces variance; bootstrap intervals on $\widehat{\text{Cost}}(\hat{\tau})$ quantify what remains. Reusing the test set both to tune $\hat{\tau}$ and to report final cost double-dips; hold the test set out of threshold selection.

4.3 Calibration: Platt scaling and isotonic regression

3.34 requires $s = \mu$. When this fails, post-hoc **calibration** fits a monotone $g : \mathbb{R} \rightarrow [0, 1]$ on held-out data and replaces s with $z := g(s)$.

Definition 3.36 (Calibration). A score s is **perfectly calibrated** if $\mathbb{P}(Y = 1 \mid s(X) = p) = p$ for all $p \in [0, 1]$. The **reliability diagram** bins observations by predicted score and plots, for

each bin B_k , the empirical positive rate $\bar{y}_k := |B_k|^{-1} \sum_{i \in B_k} y_i$ against the bin-mean $\bar{s}_k$. Perfect calibration traces the 45° diagonal.

Definition 3.37 (Brier score). For predictions $s_i \in [0, 1]$ and labels $y_i \in \{0, 1\}$, the **Brier score** is

$$\text{BS} := \frac{1}{n} \sum_{i=1}^n (s_i - y_i)^2.$$

Lemma 3.38 (Brier decomposition). Partition observations into bins $B_1, \dots, B_K$ with bin-constant predictions $s_i = \bar{s}_k$ for $i \in B_k$. Then

$$\text{BS} = \underbrace{\sum_{k=1}^K \frac{|B_k|}{n} (\bar{s}_k - \bar{y}_k)^2}_{\text{calibration loss}} + \underbrace{\sum_{k=1}^K \frac{|B_k|}{n} \bar{y}_k (1 - \bar{y}_k)}_{\text{refinement loss}}.$$

Proof. For $i \in B_k$, $s_i = \bar{s}_k$, so $(s_i - y_i)^2 = (\bar{s}_k - y_i)^2$. The bias-variance identity for binary y_i gives the within-bin mean $|B_k|^{-1} \sum_{i \in B_k} (\bar{s}_k - y_i)^2 = (\bar{s}_k - \bar{y}_k)^2 + \bar{y}_k(1 - \bar{y}_k)$ (squared bias plus binary-label variance). Summing over bins weighted by $|B_k|/n$ yields the formula. $\square$

Remark. The calibration loss is the squared per-bin gap from the reliability diagram; the refinement loss is the irreducible within-bin label noise (bounded above by $1/4$ at $\bar{y}_k = 1/2$, equal to 0 when bins are purely positive or purely negative). A perfectly calibrated score has zero calibration loss; a sharp model ($\bar{y}_k$ near 0 or 1) has small refinement loss. Continuous scores (logistic regression) need binning (typically deciles) for 3.38; without it, an extra within-bin score-variance term appears. Squared loss was rejected for **training** logistic regression (3.13: non-convex in β , vanishing gradients at the extremes); as an **evaluation** scoring rule on fitted probabilities, it is the standard quantitative companion to the reliability diagram. `yardstick::brier_class()` computes BS directly.

Setup (post-hoc calibration). Let $\mathcal{D}_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^{n_c}$ be a labeled **calibration set** disjoint from training, with raw scores $s_i := s(x_i)$. Goal: a monotone g with $g(s) \approx \mu$ that generalizes.

Definition 3.39 (Platt scaling). Fit $(a, b) \in \mathbb{R}^2$ by maximum likelihood under the Bernoulli model

$$\mathbb{P}(Y_i = 1 \mid s_i) = \sigma(a s_i + b),$$

and set the recalibrated score $z(x) := \sigma(a s(x) + b)$.

Definition 3.40 (Isotonic regression and PAV). The **isotonic** calibrator solves

$$g_{\text{iso}} := \arg \min_{g \text{ non-decreasing}} \sum_{i=1}^{n_c} (y_i - g(s_i))^2.$$

The **pool-adjacent-violators** algorithm (PAV) returns the unique solution in $O(n_c \log n_c)$: sort by s_i , initialize $g_i := y_i$, find any pair $g_j > g_{j+1}$, replace both by their pool's weighted mean; iterate until non-decreasing. Each pool average is a weighted mean of binary labels $y_i \in \{0, 1\}$, so the output $g_i \in [0, 1]$ automatically; no clipping is needed.

Remark.

- **Calibration vs. ranking.** Perfect ranking ($\text{AUC} = 1$, 3.29) does not imply calibration: place positives at $s = 0.6$, negatives at $s = 0.4$. 3.34 cares about $s = \mu$ pointwise; AUC cares only about pairwise order.
- **Platt vs. isotonic.** Platt is logistic regression (3.8) on a single feature; it assumes sigmoid-shaped error and cannot fix non-monotone distortions (overshoot mid-range, undershoot at the extremes). Isotonic is non-parametric: any monotone error is correctable, but the piecewise-constant output overfits on small $\mathcal{D}_{\text{cal}}$. Heuristic: $n_c \gtrsim 1000$ before isotonic dominates Platt; below that, Platt's two-parameter constraint regularizes.
- **Calibration set must be separate.** Training-set scores are over-confident (the model fit them), so their calibration error is training noise, not population miscalibration. Cross-validated calibration (score each training fold under a model trained on the others, pool out-of-fold scores) avoids sacrificing test data; the construction belongs to Unit 4.

Beta calibration

Between Platt's two-parameter sigmoid and isotonic's piecewise-constant step function sits beta calibration (Kull, Filho, and Flach, JMLR 2017; R package **betacal**), a smoother three-parameter mapping derived from a Beta-family link. It handles the regime where Platt is misspecified (the score-error shape is monotone but not sigmoidal) yet n_c is too small for isotonic without overfitting.

4.4 Reporting threshold-aware metrics

A bare “Accuracy 0.92” pins one **operating point**: the pair $(\tau, \text{score type})$ fixing $\hat{y} = \mathbb{1}\{s(x) \geq \tau\}$. Without both, the reader cannot reproduce the metric or assess its trade-off. ROC and PR visualize all operating points; a single metric pins one. A complete report states:

1. the cost matrix C (or its FN:FP ratio), with business justification;
2. the operating point $(\tau, \text{score type})$ and how τ was selected (Bayes formula 3.34 vs. empirical tuning 3.35);
3. the data used to tune τ , with confirmation that the test set was held out;

4. ROC (3.27) and PR (3.30) curves with the operating point marked;
5. discrete metrics at the operating point: Acc, BA, Precision, Recall, F_1 , MCC, $\widehat{\text{Cost}}$;
6. AUC (3.28) and AP (3.31) for ranking quality independent of τ .

Remark.

- **No threshold, no metric.** “Recall = 0.95, Precision = 0.40” without τ is uninterpretable: any classifier hits it by lowering τ enough. The value is the triple $(\tau, \text{Recall}, \text{Precision})$, paired with the curve showing nearby trade-offs.
- **Calibration changes the meaning of τ .** $\tau = 0.5$ on uncalibrated SVM scores is not the Bayes threshold for $c_{\text{FP}} = c_{\text{FN}}$; it is wherever 0.5 falls on the SVM scale. State whether τ is on raw scores, Platt probabilities (3.39), or isotonic ones (3.40); the same τ on three scales picks three different operating points.

5 Class Imbalance

5.1 The metric-loss disconnect under imbalance

Standard cross-entropy weights every observation equally. On imbalanced data ($P \ll N$), the negative class dominates: roughly n contributions pull $\mu(x) \rightarrow 0$ on ambiguous x , even when the analyst's metric (recall, MCC, F_1) values positives more.

Setup. The imbalance ratio is $\rho := P/N$; **imbalanced** means $\rho \ll 1$ (or $\rho \gg 1$ by symmetry). The unweighted cross-entropy (3.10) is $L_{\text{CE}}(\beta) = \sum_i \ell_i$ with $\ell_i = -y_i \log \mu_i - (1 - y_i) \log(1 - \mu_i)$ and $\mu_i = \sigma(x_i^\top \beta)$.

For the intercept-only model $\mu_i \equiv \mu$, $L_{\text{CE}}(\mu) = -P \log \mu - N \log(1 - \mu)$, and $\partial L_{\text{CE}} / \partial \mu = 0$ gives $\mu = P/(P + N)$. Pointwise: wherever features are uninformative, $\mu(x)$ is pulled toward the base rate. With $\rho = 0.01$, this baseline is 0.01, and $\tau = 0.5$ classifies nearly all such x as negative regardless of the FP:FN ratio.

Definition 3.41 (Class-weighted cross-entropy). For positive weights $w_+, w_- > 0$,

$$L_w(\beta) := \sum_{i=1}^n [w_+ y_i + w_- (1 - y_i)] \ell_i.$$

The **balanced** choice $w_+ = n/(2P)$, $w_- = n/(2N)$ gives equal total weight to each class; the same calculus then yields intercept-only minimizer $\mu = 1/2$.

Remark. Reweighting and oversampling are loss-equivalent: duplicating each positive w_+/w_- times reproduces L_w exactly. Reweighting avoids both the variance of duplicates and the distortion of synthetic samples, but cannot fix calibration: μ now estimates $\mathbf{P}(Y = 1 \mid X = x)$ under the **reweighted** distribution, so deployment still needs a prior correction (treated below).

5.2 Resampling: oversampling, undersampling, SMOTE

A second remedy modifies the training set rather than the loss.

Definition 3.42 (Resampling methods). Three balanced-set constructions on the minority class:

- **Random oversampling.** Sample $N - P$ positives with replacement and append; size $2N$.
- **Random undersampling.** Sample P negatives without replacement, discard the rest; size $2P$.
- **SMOTE.** For each positive x_i , find its k nearest minority-class neighbors. Pick a neighbor

x_j uniformly at random and create a synthetic positive

$$\tilde{x} = x_i + \alpha (x_j - x_i), \quad \alpha \sim \text{Uniform}[0, 1],$$

labeled $\tilde{y} = 1$; repeat until balanced.

Remark.

- **Decision-boundary effects.** Oversampling shifts the boundary toward the majority without changing the positive region's geometry (duplicates are coincident). Undersampling discards majority information and inflates variance on small data. SMOTE places synthetic positives on segments between nearest minority neighbors, smoothing the boundary into majority territory; it also distorts categorical features into meaningless fractional values.
- **Resampling leakage.** Resampling must occur **after** the train/test split and **inside** CV folds. SMOTE on the full dataset creates synthetic positives that interpolate between train and test points; the test set then contains training-derived structures, and reported performance over-estimates. Inside a CV fold: fit on the resampled training portion, evaluate on the original validation portion.

5.3 Threshold adjustment vs. retraining

Reweightings (3.41) and resampling (3.42) modify the training data; threshold adjustment (§4) only changes τ . At the population level, both yield the same Bayes-optimal decision.

Lemma 3.43 (Equivalence of threshold tuning and class weighting). Assume the model class attains the population minimum of its training loss. Both procedures predict identically on every x :

1. train unweighted cross-entropy; predict $\mathbb{1}\{\mu(x) \geq \tau^*\}$ with $\tau^* = c_{\text{FP}}/(c_{\text{FP}} + c_{\text{FN}})$;
2. train weighted cross-entropy with $w_+/w_- = c_{\text{FN}}/c_{\text{FP}}$; predict $\mathbb{1}\{\mu_w(x) \geq 0.5\}$.

Proof. At the population minimum, unweighted CE recovers $\mu(x) := \mathbf{P}(Y = 1 \mid X = x)$ (3.11). The weighted population loss at x is $-w_+\mu(x) \log \hat{\mu} - w_-(1 - \mu(x)) \log(1 - \hat{\mu})$; differentiating in $\hat{\mu}$ and setting to zero gives

$$\mu_w(x) = \frac{w_+ \mu(x)}{w_+ \mu(x) + w_- (1 - \mu(x))}.$$

Thresholding $\mu_w(x) \geq 0.5$ rearranges to $\mu(x) \geq w_-/(w_+ + w_-)$. Setting $w_+/w_- = c_{\text{FN}}/c_{\text{FP}}$:

$$\frac{w_-}{w_+ + w_-} = \frac{1}{1 + w_+/w_-} = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = \tau^*.$$

Both procedures therefore predict $\hat{y} = 1 \iff \mu(x) \geq \tau^*$. □

Remark.

- **Threshold tuning is a leaner approach.** It adjusts only a single scalar τ , avoiding training-data distortion and information loss. It requires **calibrated** scores: logistic regression provides them under correct specification, but boosted trees and SVMs do not without post-hoc correction (3.39, 3.40).
- **When retraining still helps.** Severe imbalance ($\rho \lesssim 10^{-3}$): the optimizer rarely sees positives, and reweighting amplifies the positive gradient. Tree-based or non-probabilistic models lacking calibration, when no calibration split is affordable. Too few positives for local structure: SMOTE adds support where threshold tuning on sparse positives is unstable.

5.4 Prior correction under deployment shift

Resampling and reweighting shift the training prior. A model trained on a balanced set ($\pi_t = 0.5$) and deployed at $\pi_d \neq 0.5$ over-predicts positives; the correction is a one-line logit shift.

Lemma 3.44 (Prior correction). Let $\mu_t(x)$ be the model's posterior under training prior π_t . Assume the class-conditional density $f(x | Y)$ is the same on training and deployment (only the

prior shifts). The deployment posterior is

$$\mu_d(x) = \sigma \left(\text{logit}(\mu_t(x)) + \log \frac{\pi_d/(1 - \pi_d)}{\pi_t/(1 - \pi_t)} \right).$$

Proof. Bayes' rule gives $\mu(x)/(1 - \mu(x)) = [f(x | Y = 1)/f(x | Y = 0)] \cdot \pi/(1 - \pi)$, so

$$\text{logit } \mu(x) = \log \frac{f(x | Y = 1)}{f(x | Y = 0)} + \log \frac{\pi}{1 - \pi}.$$

The likelihood-ratio term is assumption-invariant, so subtracting training from deployment cancels it:

$$\text{logit } \mu_d(x) - \text{logit } \mu_t(x) = \log \frac{\pi_d}{1 - \pi_d} - \log \frac{\pi_t}{1 - \pi_t}.$$

Applying σ to both sides recovers the stated $\mu_d(x)$. □

Remark.

- **Pure arithmetic correction.** Optimize on whatever class balance simplifies training; then shift the logits at deployment by the difference in prior log-odds. Reweighting and resampling implicitly assume $\pi_d = 0.5$; deploying a balanced-trained model at $\pi_d = 0.01$ without correction over-predicts positive odds by nearly $100\times$.
- **Covariate-shift assumption.** The assumption that $f(x \mid Y)$ is unchanged is strong. If the deployment class-conditional distribution differs (covariate shift on X , or concept drift in $Y \mid X$), the likelihood-ratio term itself moves and prior correction fails. Validity checks were the subject of Unit 1; remediation lies outside Unit 3's scope.

6 Implementation Tour

Three moves implement the theory of §§1–5: fit a probabilistic classifier (tidymodels), evaluate ranking and threshold-aware metrics (yardstick), tune the threshold under the operating cost. A from-scratch IRLS closes the loop in twenty lines. Examples use `ISLR2::Default` (10 000 credit-card holders, $\sim 3.3\%$ defaulters).

6.1 tidymodels workflow

Five objects: **split** (partition), **recipe** (preprocessing), **model spec** (algorithm), **workflow** (recipe + spec), **fit**. `augment()` appends class predictions `.pred_class` and class probabilities `.pred_Yes`, `.pred_No` to a held-out frame.

```
1 library(tidymodels); library(ISLR2)
2 data(Default)
3
4 set.seed(1)
5 split <- initial_split(Default, prop = 0.8, strata = default)
6 train <- training(split); test <- testing(split)
7
8 rec <- recipe(default ~ ., data = train) |>
9   step_normalize(all_numeric_predictors()) |>
10  step_dummy(all_nominal_predictors())
11
12 mod <- logistic_reg() |> set_engine("glm")
13 wf  <- workflow() |> add_recipe(rec) |> add_model(mod)
14 fit <- fit(wf, data = train)
15
```

```
16 pred <- augment(fit, new_data = test)           # adds .pred_class, .pred_Yes
17 head(pred[, c("default", ".pred_class", ".pred_Yes")], 5)
```

```
## # A tibble: 5 x 3
##   default .pred_class .pred_Yes
##   <fct>    <fct>        <dbl>
## 1 No      No              0.00144
## 2 No      No              0.0000106
## 3 No      No              0.0000759
## 4 No      No              0.0000231
## 5 No      No              0.00171
```

Remark. `strata = default` preserves the imbalance ratio across the split. `step_normalize` is harmless for `glm` but required for `glmnet`; keeping normalization in the recipe (not in upstream feature engineering) reuses the `train` parameters on `test` without leakage.

6.2 Evaluation and threshold tuning

`yardstick` reports threshold-free (AUC, AP) and threshold-fixed (accuracy, F_1 , MCC) metrics from one prediction frame. `event_level = "second"` treats the second factor level (**Yes**) as positive.


```
1 # Threshold-free ranking metrics
2 roc_auc(pred, default, .pred_Yes, event_level = "second")
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 roc_auc binary      0.953
```

```
1 pr_auc (pred, default, .pred_Yes, event_level = "second")
```

```
## # A tibble: 1 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>       <dbl>
## 1 pr_auc  binary      0.448
```

```
1 # Threshold-fixed metrics at default tau = 0.5 (positive class = "Yes")
2 metric_set(accuracy, bal_accuracy, mcc, f_meas)(
3   pred, truth = default, estimate = .pred_class, event_level = "second"
4 )
```

```
## # A tibble: 4 x 3
```

```
##      .metric      .estimator .estimate
##      <chr>        <chr>      <dbl>
## 1 accuracy      binary      0.966
## 2 bal_accuracy  binary      0.596
## 3 mcc           binary      0.326
## 4 f_meas        binary      0.295
```

```
1  # Empirical threshold sweep under cost (c_FP = 1, c_FN = 50):
2  # vectorized  $O(n_v \log n_v)$  sort-and-sweep matching the theory of \ref{def:empirical-cost}
3  ord      <- order(pred$.pred_Yes, decreasing = TRUE)
4  y_sorted <- pred$default[ord] == "Yes"
5  TP <- cumsum( y_sorted)
6  FP <- cumsum(!y_sorted)
7  FN <- sum(y_sorted) - TP
8  costs    <- 1 * FP + 50 * FN                                # c_FP = 1, c_FN = 50
9  opt_idx  <- which.min(costs)
10
11 c(tau_hat      = pred$.pred_Yes[ord][opt_idx],
12   cost_min      = costs[opt_idx],
13   cost_at_0.5   = sum(pred$.pred_Yes >= 0.5 & pred$default == "No") +
```

```
14 50 * sum(pred$.pred_Yes < 0.5 & pred$default == "Yes"))
```

```
##      tau_hat      cost_min  cost_at_0.5  
## 1.274055e-02 4.980000e+02 2.860000e+03
```

Remark. The sweep implements 3.35 at every distinct score. In production, $\hat{\tau}$ is tuned on a validation split (`validation_split()` or `vfold_cv()`) before the final test report; tuning on `test` above is a shortcut for exposition. For production, wrap $\widehat{\text{Cost}}$ as a custom `yardstick` metric so it composes with `metric_set()` and `tune_grid()`.

Modern parsnip-native threshold tuning

The production idiom is `probably::threshold_perf()`, which sweeps a yardstick metric over a grid of τ values from a prediction frame and returns a long-format tibble for plotting or argmin selection; `tune::tune_grid()` composes with it via the `post =` argument so τ is tuned within the resampling loop. The from-scratch sort-and-sweep above remains pedagogically valuable because it exposes the $O(n_v \log n_v)$ structure of 3.35 that `probably` hides behind a uniform grid.

6.3 IRLS from scratch

Each Newton iteration of 3.14 is a weighted least-squares fit on the working response $z_i := \eta_i + (y_i - \mu_i)/[\mu_i(1 - \mu_i)]$. Twenty lines match `glm()` to machine precision.

```

1 irls <- function(X, y, max_iter = 50, tol = 1e-8) {
2    beta <- rep(0, ncol(X))
3    for (k in seq_len(max_iter)) {
4      eta <- as.vector(X %*% beta)
5      mu  <- 1 / (1 + exp(-eta))           # sigmoid
6      mu  <- pmin(pmax(mu, 1e-15), 1 - 1e-15) # clip to avoid div-by-zero in z
7      W   <- mu * (1 - mu)                 # diagonal weights, length n
8      z   <- eta + (y - mu) / W            # working response
9      # W * X recycles W column-wise: equivalent to diag(W) %*% X
10     beta_new <- solve(crossprod(X, W * X), crossprod(X, W * z))
11     if (max(abs(beta_new - beta)) < tol) break
12     beta <- as.vector(beta_new)
13   }
14   list(beta = beta, iter = k)
15 }
16
17 # Smoke test: recover glm() coefficients on simulated data

```

```
18 set.seed(1)
19 n <- 1000
20 X <- cbind(1, matrix(rnorm(n * 3), n, 3))
21 beta_true <- c(-2, 0.8, -0.5, 0.3)
22 y <- rbinom(n, 1, plogis(X %*% beta_true))
23
24 fit_irls <- irls(X, y)
25 fit_glm <- glm(y ~ X[, -1], family = binomial())
26 cbind(scratch = fit_irls$beta, glm = unname(coef(fit_glm)))
```

```
##           scratch           glm
## [1,] -1.8231614 -1.8231614
## [2,]  0.6957335  0.6957335
## [3,] -0.3838313 -0.3838313
## [4,]  0.2978444  0.2978444
```

Remark. Diagonal weights $W = \text{diag}(\mu_i(1 - \mu_i))$ shrink toward zero as $\mu_i \rightarrow 0$ or 1 , so confident points barely move the next step. Clipping μ to $[10^{-15}, 1 - 10^{-15}]$ avoids $W = 0$ (and **Inf** in z) when μ rounds to $\{0, 1\}$; `glm.fit` uses analogous bounds. Beyond clipping, failure signals either (i) perfect separation or (ii) collinearity ($X^\top W X$ rank-deficient). `glm()` reports (i) as “**fitted probabilities numerically 0 or 1 occurred**”.

7 Pitfalls and Misapplications

Each entry pairs a misuse with a diagnostic and a runnable demo. This catalog collects what §§1–5 forbid and seed the trap bank for the final project.

7.1 Accuracy on imbalanced data

Accuracy = 0.99 on a 1%-positive dataset is meaningless: “predict 0” achieves it with zero recall. Accuracy is the default in introductory treatments and in `tidymodels`’ `accuracy()`, so the trap is one keystroke away.

```
1 set.seed(1)
2 n <- 1000
3 y <- rbinom(n, 1, 0.01)           # 1% positives
4 yhat <- rep(0L, n)               # always predict 0
```

```

5 P <- sum(y); N <- n - P
6 acc <- mean(yhat == y)
7 ba <- 0.5 * (sum(yhat == 0 & y == 0)/N + sum(yhat == 1 & y == 1)/P)
8 c(base_rate = P/n, accuracy = acc, recall = 0, balanced_acc = ba)

```

```

##      base_rate      accuracy      recall balanced_acc
##      0.014         0.986         0.000         0.500

```

Diagnostic. Compare accuracy against the base-rate baseline $1 - P/n$ (intercept-only minimizer, §5.1); equality means the classifier learned nothing.

Fix. Balanced accuracy (3.20), MCC (3.21), or cost-weighted error (3.23) tied to the business asymmetry.

7.2 ROC over-optimistic under extreme imbalance

AUC near 1 hides poor precision when $N/P \gg 1$. ROC is the textbook curve, looks symmetric, and reports favorably even when most positive predictions are false.

```

1 P <- 100; N <- 9900                                # 1% positive rate
2 TPR <- 1.0; FPR <- 0.02                             # excellent on ROC
3 TP <- TPR * P; FP <- FPR * N
4 precision <- TP / (TP + FP)

```

```
5 c(TPR = TPR, FPR = FPR, precision = precision, false_pos_count = FP)
```

##	TPR	FPR	precision	false_pos_count
##	1.0000000	0.0200000	0.3355705	198.0000000

Diagnostic. Report AP alongside AUC; mark the operating point on a PR curve. Under severe imbalance, precision $\rightarrow 0$ as $N/P \rightarrow \infty$ at any fixed FPR > 0 (3.33); prefer PR/AP (3.31) as the primary curve.

7.3 Threshold defaulted to 0.5

$\tau = 0.5$ is Bayes-optimal only when $c_{FP} = c_{FN}$ (3.34). It is the default in `predict_class/glm`, the sigmoid midpoint, and intuitive; under asymmetric costs it is wrong.

```
1 set.seed(1)
2 n <- 1000
3 mu <- runif(n, 0, 1) # true conditional probability
4 y <- rbinom(n, 1, mu)
5 score <- pmin(pmax(mu + rnorm(n, 0, 0.05), 0), 1) # near-calibrated
6
7 emp_cost <- function(tau, cfp = 1, cfn = 50) {
8   yhat <- score >= tau
```



```

9   cfp * sum(yhat & y == 0) + cfn * sum(!yhat & y == 1)
10 }
11 taus <- seq(0, 1, 0.01)
12 costs <- vapply(taus, emp_cost, numeric(1))
13 c(cost_at_0.5 = emp_cost(0.5),
14   cost_tuned  = min(costs),
15   tau_star    = taus[which.min(costs)])

```

```

## cost_at_0.5  cost_tuned  tau_star
##          6626.00      481.00      0.03

```

Diagnostic. Tune τ on validation data under the cost ratio; $\tau^* \neq 0.5$ unless costs are symmetric. Use the Bayes formula (3.34) when calibrated, the empirical sweep (3.35) otherwise.

7.4 SMOTE before the train/test split

SMOTE on the full dataset puts synthetic positives in the test set; many interpolate between training rows, so the test set reports the model’s training fit. Tutorials show “balance, then split” for visual cleanliness; that is the temptation.

```

1 library(themis)
2 default_tagged <- Default |> mutate(row_id = seq_len(nrow(Default)))

```

```

3
4 # WRONG: SMOTE on full data, then split
5 set.seed(1)
6 prepped <- recipe(default ~ ., data = default_tagged) |>
7   update_role(row_id, new_role = "id") |>
8   step_dummy(student) |>
9   step_smote(default) |> prep() |> bake(new_data = NULL)
10 test_wrong <- testing(initial_split(prepped, prop = 0.8))
11
12 # RIGHT: split first; balance the training subset only (step_smote in the recipe)
13 set.seed(1)
14 test_right <- testing(initial_split(Default, prop = 0.8, strata = default))
15
16 c(orig_pos_rate      = mean(Default$default == "Yes"),
17   test_pos_wrong     = mean(test_wrong$default == "Yes"),
18   test_pos_right     = mean(test_right$default == "Yes"),
19   synth_share_test   = mean(is.na(test_wrong$row_id)))

```

##	orig_pos_rate	test_pos_wrong	test_pos_right	synth_share_test
##	0.0333000	0.5045255	0.0355000	0.4887510

Diagnostic. The test positive rate should match deployment; tag rows by origin to surface synthetic leakage.

Fix. Place `themis::step_smote()` inside the recipe so `fit_resamples()` and final-fit regenerate synthetic positives from the appropriate training subset only.

7.5 SMOTE outside the CV folds

SMOTE applied once before cross-validation makes every fold share the same synthetic structure: the inflated set is identical across folds, validation rows can contain synthetic positives derived from training, and per-fold variance shrinks artificially. Pre-resampling looks cheaper than per-fold resampling; that is the temptation.

```
1 set.seed(1)
2 sp <- initial_split(Default, prop = 0.8, strata = default)
3 tr <- training(sp)
4
5 # RIGHT: step_smote inside the recipe -> applied to each training fold
6 rec_in <- recipe(default ~ ., data = tr) |>
7   step_dummy(all_nominal_predictors()) |> step_smote(default)
8
```

```
9 # WRONG: balance the entire training set once, then run CV on it
10 balanced <- recipe(default ~ ., data = tr) |>
11   step_dummy(all_nominal_predictors()) |> step_smote(default) |>
12   prep() |> bake(new_data = NULL)
13 rec_out <- recipe(default ~ ., data = balanced)
14
15 mod <- logistic_reg() |> set_engine("glm")
16 wf_in <- workflow() |> add_recipe(rec_in) |> add_model(mod)
17 wf_out <- workflow() |> add_recipe(rec_out) |> add_model(mod)
18
19 set.seed(2)
20 res_in <- fit_resamples(wf_in, vfold_cv(tr, v = 5, strata = default),
21   metrics = metric_set(bal_accuracy))
22 res_out <- fit_resamples(wf_out, vfold_cv(balanced, v = 5),
23   metrics = metric_set(bal_accuracy))
24
25 list(inside = collect_metrics(res_in),
26   outside = collect_metrics(res_out))
```

```
## $inside
## # A tibble: 1 x 6
```

```
##      .metric      .estimator  mean      n std_err .config
##      <chr>        <chr>      <dbl> <int>   <dbl> <chr>
## 1 bal_accuracy binary      0.873     5  0.0122 pre0_mod0_post0
##
## $outside
## # A tibble: 1 x 6
##      .metric      .estimator  mean      n std_err .config
##      <chr>        <chr>      <dbl> <int>   <dbl> <chr>
## 1 bal_accuracy binary      0.898     5 0.00250 pre0_mod0_post0
```

Diagnostic. Outside-CV scores are more optimistic and lower-variance than honest inside-CV estimates.

Fix. Place `themis::step_smote()` inside the recipe; `fit_resamples()` refits the recipe per training fold, leaving validation rows untouched.

7.6 Tree-based scores treated as calibrated probabilities

Random-forest and boosted-tree scores are not calibrated: vote/leaf averages cluster at discrete values and concentrate away from 0 and 1 (3.36). Logistic regression’s MLE is calibrated by construction under correct specification. Both output a number in $[0, 1]$, and the analyst silently reads “probability”

off either.

```
1 library(probably); library(ranger)
2 set.seed(1)
3 sp <- initial_split(Default, prop = 0.6, strata = default)
4 tr <- training(sp); te <- testing(sp)
5
6 fit_lr <- workflow() |> add_recipe(rec) |>
7   add_model(logistic_reg() |> set_engine("glm")) |> fit(tr)
8 fit_rf <- workflow() |> add_recipe(rec) |>
9   add_model(rand_forest(trees = 500) |> set_engine("ranger") |>
10     set_mode("classification")) |> fit(tr)
11
12 pred_lr <- augment(fit_lr, te); pred_rf <- augment(fit_rf, te)
13
14 # Reliability gap per decile bin: empirical positive rate minus mean score.
15 # Empty bins explicitly return NA via `default = NA_real_`; an NA in
16 # the random-forest column is itself a diagnostic signal (the model
17 # never predicts in that score range).
18 reliab <- function(p, K = 10) {
19   bin      <- cut(p$.pred_Yes, breaks = seq(0, 1, length.out = K + 1), include.lowest = TRUE)
```

```

20   emp_rate <- tapply(as.integer(p$default == "Yes"), bin, mean, default = NA_real_)
21   pred_avg <- tapply(p$.pred_Yes, bin, mean, default = NA_real_)
22   emp_rate - pred_avg
23 }
24 round(cbind(logistic = reliab(pred_lr), random_forest = reliab(pred_rf)), 3)

```

```

##           logistic random_forest
## [0,0.1]      0.001      -0.007
## (0.1,0.2]   -0.011       0.025
## (0.2,0.3]   -0.060       0.149
## (0.3,0.4]    0.030       0.162
## (0.4,0.5]    0.035       0.185
## (0.5,0.6]    0.002       0.150
## (0.6,0.7]   -0.049      -0.048
## (0.7,0.8]    0.148        NA
## (0.8,0.9]   -0.122        NA
## (0.9,1]     -0.464        NA

```

Diagnostic. Reliability diagram (3.36) on a held-out split; large per-bin gaps and missing extreme bins (RF rarely predicts > 0.7) signal miscalibration.

Fix. Platt scaling (3.39) for sigmoid-shaped error, isotonic (3.40) for arbitrary monotone error, fit

on a separate calibration split. The `probably` package provides `cal_estimate_logistic()` and `cal_estimate_isotonic()`.

7.7 Threshold tuned on training data

Tuning τ on training scores and reporting test metrics double-dips: the training set fitted both the score and the threshold, biasing the reported test cost downward. Training data is the largest, easiest source of scores; the lowest-effort default.

```
1 set.seed(1); n <- 1000
2 X <- matrix(rnorm(n * 3), n, 3)
3 beta_true <- c(-2, 0.8, -0.5, 0.3)
4 y <- rbinom(n, 1, plogis(cbind(1, X) %*% beta_true))
5
6 i_train <- 1:600; i_val <- 601:800; i_test <- 801:1000
7 fit <- glm(y[i_train] ~ X[i_train, ], family = binomial())
8 score <- function(idy) as.vector(plogis(cbind(1, X[idy, ]) %*% coef(fit)))
9
10 emp_cost <- function(s, yy, tau, cfp = 1, cfn = 10) {
11   yhat <- s >= tau
12   cfp * sum(yhat & yy == 0) + cfn * sum(!yhat & yy == 1)
```



```
13 }
14 tune <- function(s, yy) {
15   taus <- sort(unique(s))
16   taus[which.min(vapply(taus, emp_cost, numeric(1), s = s, yy = yy))]
17 }
18 tau_tr <- tune(score(i_train), y[i_train])
19 tau_va <- tune(score(i_val), y[i_val])
20 c(tau_train = tau_tr, tau_val = tau_va,
21   test_cost_with_train_tau = emp_cost(score(i_test), y[i_test], tau_tr),
22   test_cost_with_val_tau    = emp_cost(score(i_test), y[i_test], tau_va))
```

```
##           tau_train           tau_val test_cost_with_train_tau
##           0.09208217           0.12679059           138.00000000
## test_cost_with_val_tau
##           127.00000000
```

Diagnostic. A three-way split (train / validation / test) eliminates the leakage by construction; when data is scarce, cross-validate the threshold instead. Final test reports must use a τ chosen without seeing test labels.